

AUTOMATIC-FRAUD-DETECTION-PROJECT

90 min.

1. Context

Fraud is a huge issue among financial institutions. In the EU in 2019, the [European Central Bank](#) estimated that fraudulent credit card transactions amounted more €1 billion! 😬

AI can really help solve this issue by detecting fraudulent payments in a very precise manner. This usecase is actually now one of the most famous one among Data Scientists.

However, even though we managed to build powerful algorithms, the hard thing is now to use them in production. This means predict fraudulent payment in real-time and respond appropriately.

2. Project Goals

Business has come to your team because they need to:

G1. Fraud Detection and Notification

Get notified once a fraud is detected. They told you that they just need a notification though.

G2. Daily Payments and Fraud Checking

Once every morning, they need to check all the payments and frauds that happened during the previous day.

3. Where to get data

To achieve this project, you will need to get access to some data! Here are two data sources that you can use:

- [Fraudulent Payments full dataset](#)
 - This dataset contains a large amount of payments labelled as fraudulent or not
 - Use this to create your algorithm!
- [Real-time payment API](#)
 - This API retrieves real-time payments (see the `/current-transactions` endpoint in the Space documentation)
 - Use this to create real-time predictions
 - The API gets updated **every minute**

4. Deliverables

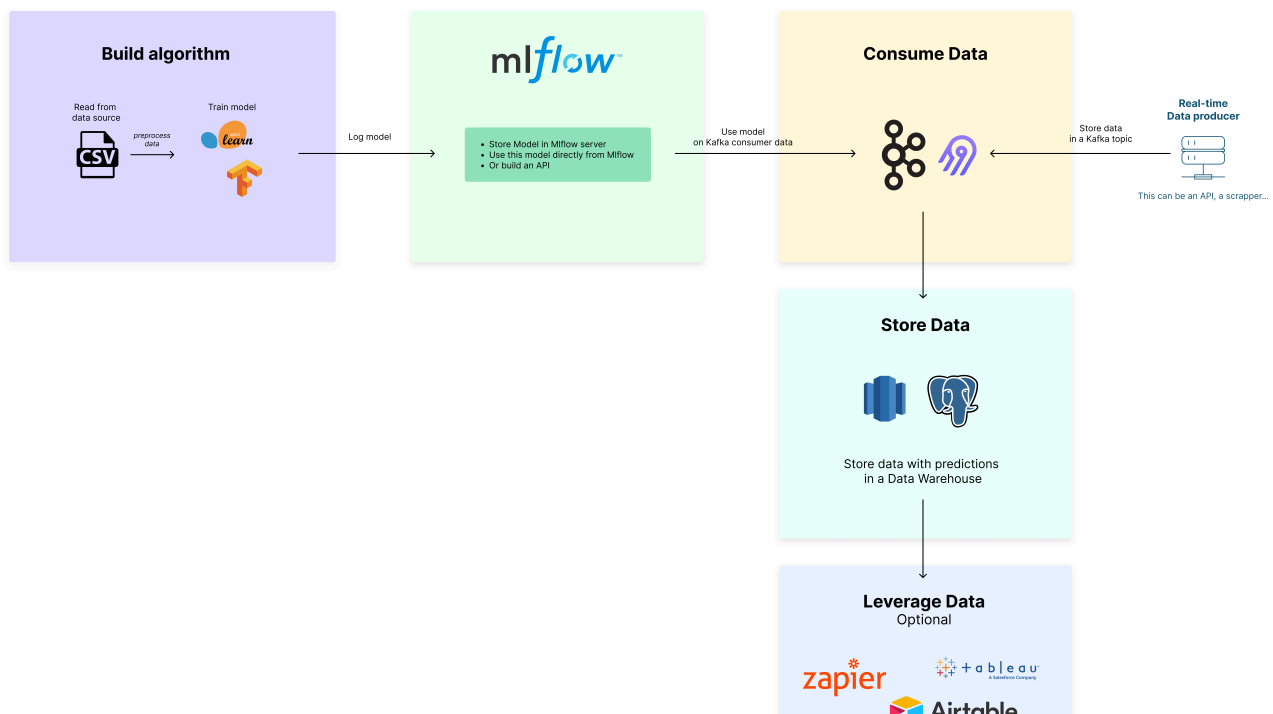
To complete this project, you will need to produce :

D1. The Data Infrastructure Diagram

A **schema** of the infrastructure you chose to build and **why** you built it that way

- This can be in a Powerpoint, Word document
- You can get inspiration from the below picture

Data Infrastructure Example



D2. The source code

The source code of all elements necessary to build your infrastructure

D3. A video capture

A **video** of your working infrastructure on an example. You can use [Vidyard](#) to do so.

5. Tips

To help you in your task, we would like to give you a few tips on how to tackle that project.

5.1. How can I build the algorithm?

To build you algorithm, you can use **any** library (`sklearn` , `tensorflow`), [APIs](#) or even [No-code](#) tools.

If you don't want to build the algorithm yourself, feel free to use *AmazonML*, which is a great API tool. Check out our [online-course on the matter](#)

Remember to build something that is reusable! Especially in terms of preprocessing, using an algorithm is not only about using a `.predict()`.

IMPORTANT | The most important part is the Data Pipeline - NOT THE ML Algorithm

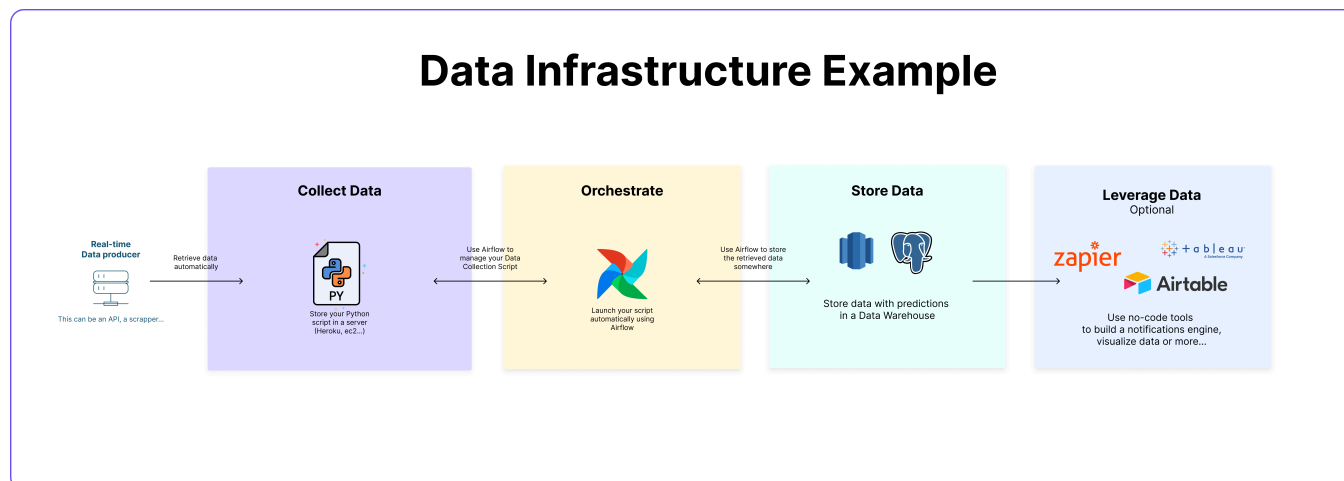
No matter what you do, the ML algorithm comes second after the Data Pipeline which should be your main priority.

5.2. I don't know where to start

First things first, in this project you need to at least:

- Train an algorithm
- Stage it into production
- Store real-time data in a db

This is our recommendation on where to start:



IMPORTANT

The above example is just a suggestion, you **can deviate** from this infrastructure, and most likely there are alternatives that might be smarter, easier to implement and more efficient. The only minimal elements we need to have are:

- An element collecting & storing data
- An element consuming data
- An ETL (or ELT) process

5.3. How do I split work among my teammates?

Working together is key here! You can split your work several ways, but here is a suggestion:

- One team member can train the algorithm
- One team member can build MLflow infrastructure
- One team member can create the real-time data ingestion pipeline